

Grid

Maider Díaz Salinas
maiderdiaz@egibide.org

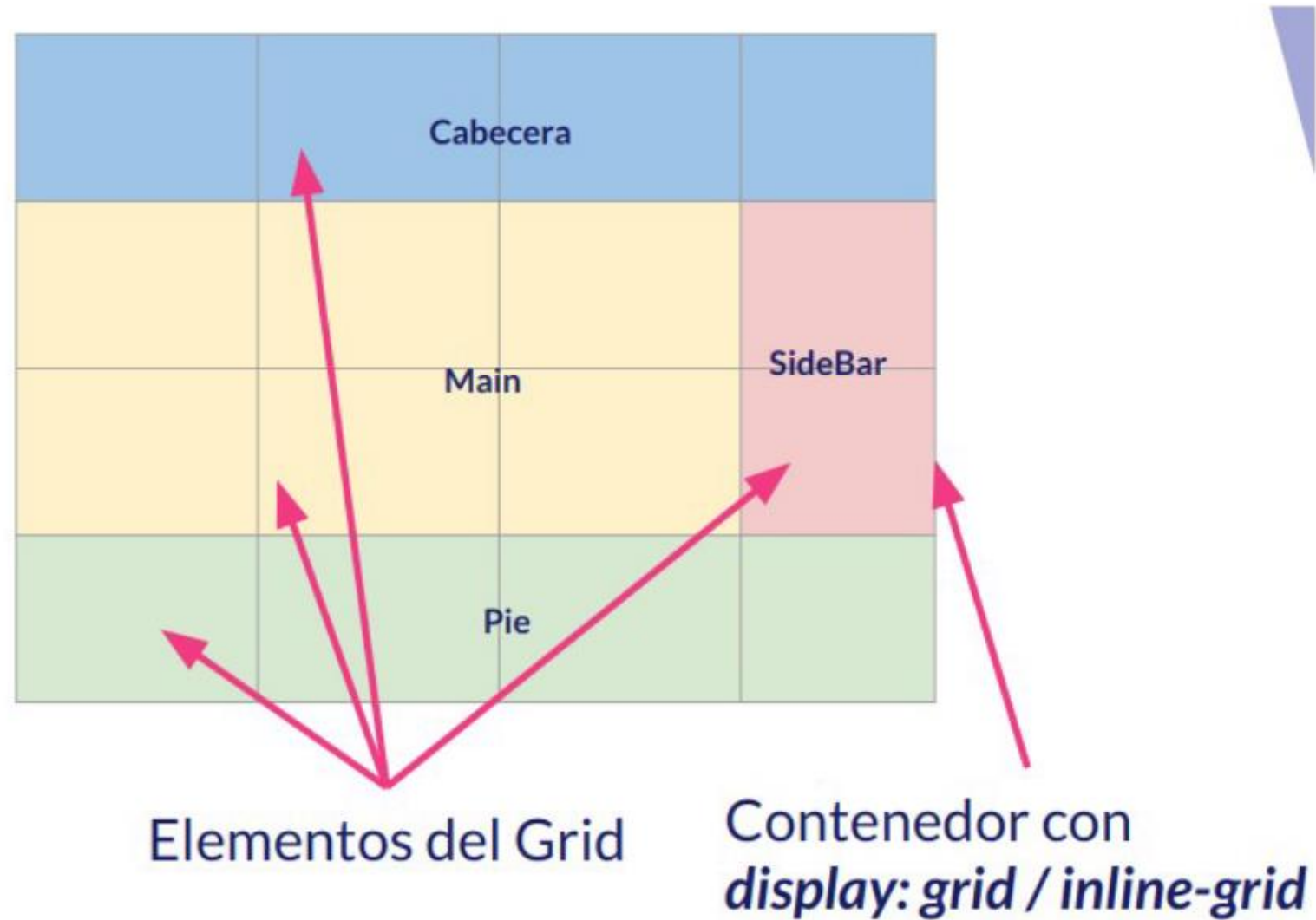
Contenido

- ¿Qué es Grid?
- Contenedor y sus elementos
- ¿Cómo crear la rejilla?
- Propiedades del contenedor
- Propiedades de los elementos

¿Qué es grid?

Es un método que asigna un elemento como contenedor (**container**) y se le asocia una rejilla, para luego ir colocando cada uno de sus elementos (**items**) en esta rejilla.

Items vs Container



Container

Un elemento con la propiedad **display** con valor **grid** o **inline-grid**, se convierte automáticamente en un container y sus hijos en ítems.

¿Cómo crear la rejilla?

Para crear la rejilla deberemos establecer cuantas **columnas** y cuantas **filas** tiene esta, además de los **espacios** entre columnas y filas.

Columnas y filas

grid-template-columns: Estable cuantas columnas y el ancho de estas.

grid-template-rows: Establece cuantas filas y el ancho de estas.

grid-template: $\text{grid-template-rows} / \text{grid-template-columns}$

Columnas y filas

/ Tres columnas que se reparten el 100% del contenedor*/*

`grid-template-columns: 20% 50% 30%;`

/ Cuatro columnas. Tres de tamaño fijo 100px y la otra ocupa ~~el espacio libre restante~~ */ lo que ocupe su contenido*/*

`grid-template-columns: 100px auto 100px 100px;`

/ Cuatro columnas. Todas ~~con un tamaño igual~~ */ ocupan lo que ocupe su contenido*/*

`grid-template-columns: auto auto auto auto;`

/ Tres columnas cada una con nombre (entre []). Dos con tamaño fijo y la otra ocupando ~~el espacio restante~~ */ lo que ocupe su contenido*/*

`grid-template-columns: [id] 100px [nombre] 300px [apellidos] auto;`

Columnas y filas

código

```
/* Cuatro columnas. Tres de 20% con nombre col-start. Y la último que ocupará el  
resto del espacio libre */ lo que ocupe su contenido*/  
grid-template-columns: repeat(3, 20% [col-start]) auto;  
/* Cuatro columnas. Una de tamaño fijo y las demás se reparten el espacio libre en 5  
partes de la siguiente manera (2+1+2) */  
grid-template-columns: 2fr 100px 1fr 2rf;
```

Columnas y filas

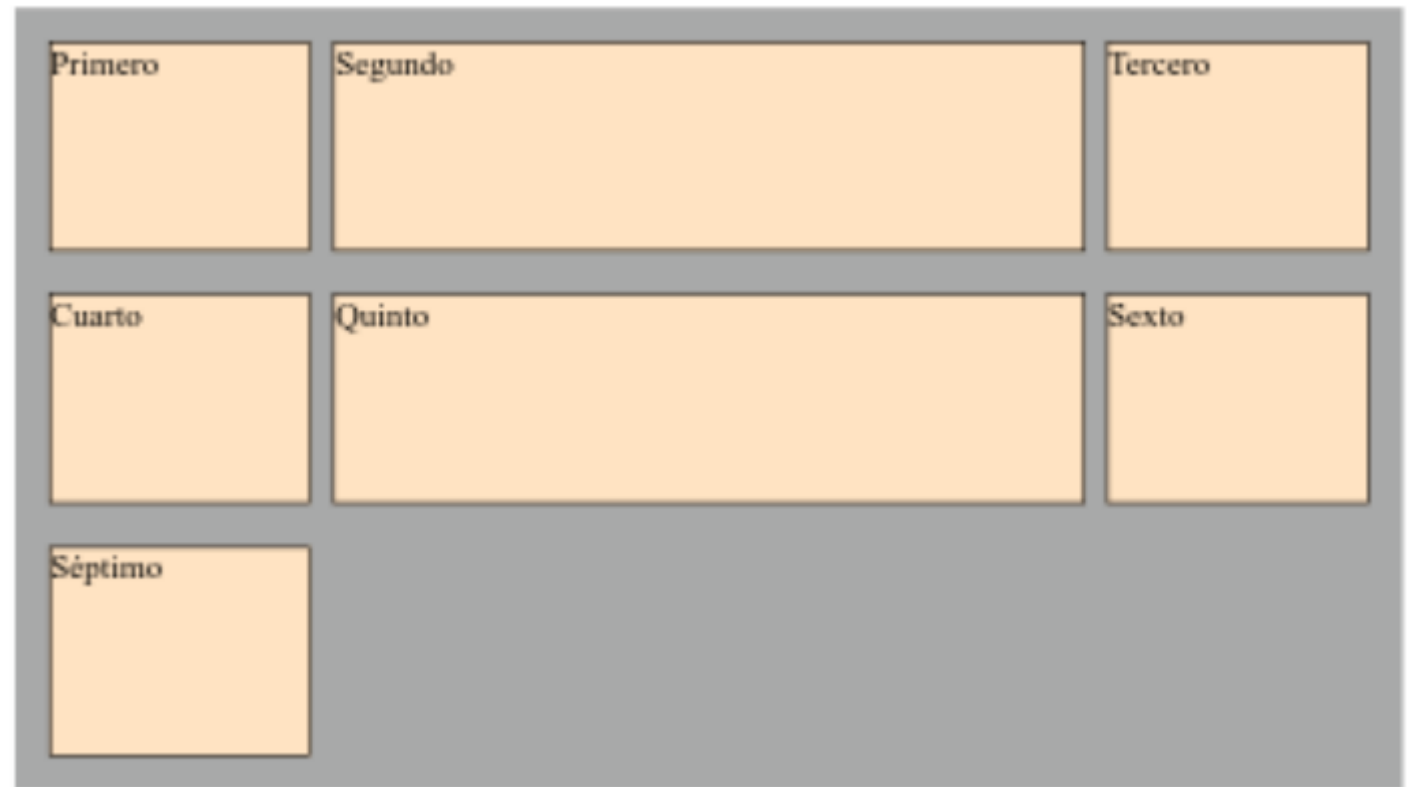
column-gap: Estable el espacio entre columnas

row-gap: Establece el espacio entre filas

Ejercicio

Partiendo del siguiente HTML crear la página de la imagen.

```
<div class="container">  
  <div>Primero</div>  
  <div>Segundo</div>  
  <div>Tercero</div>  
  <div>Cuarto</div>  
  <div>Quinto</div>  
  <div>Sexto</div>  
  <div>Séptimo</div>  
</div>
```



Propiedades del contenedor

- justify-items
- align-items
- place-items
- justify-content
- align-content

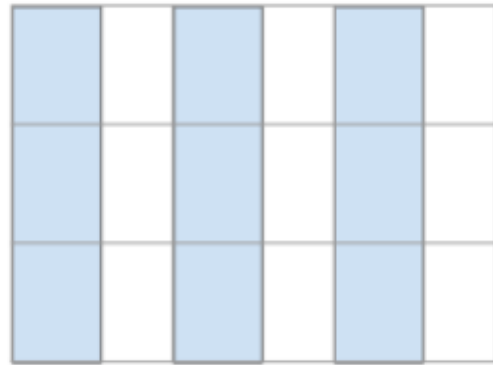
Alineación horizontal

Por defecto los elementos del grid ocupan todo el ancho de la celda (**stretch**), pero se pueden dar otras alineaciones, dándole diferentes valores a la propiedad **justify-items**.

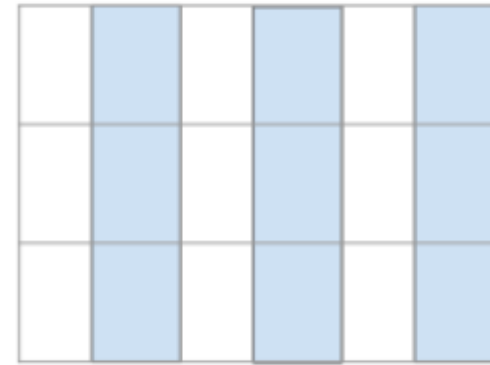
- **start**
- **end**
- **center**
- **stretch**

Alineación horizontal

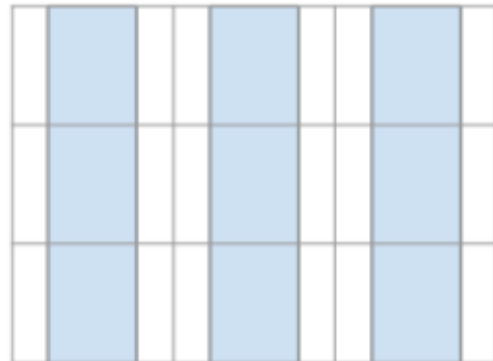
start



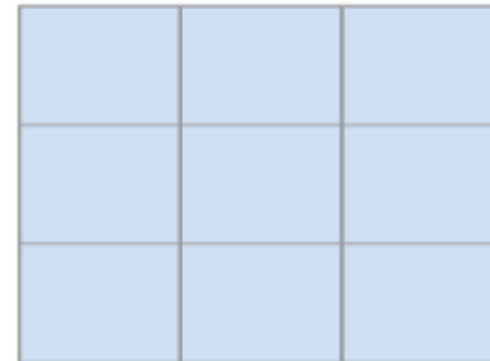
end



center



stretch



Alineación vertical

Al igual que con la alineación horizontal, por defecto los elementos del grid ocupan todo el alto de la celda (**stretch**), pero se pueden dar otras alineaciones, dándole diferentes valores a la propiedad **align-items**.

- **start**
- **end**
- **center**
- **stretch**

Alineación horizontal

start

end

center

[illegible]

stretch

Alineación

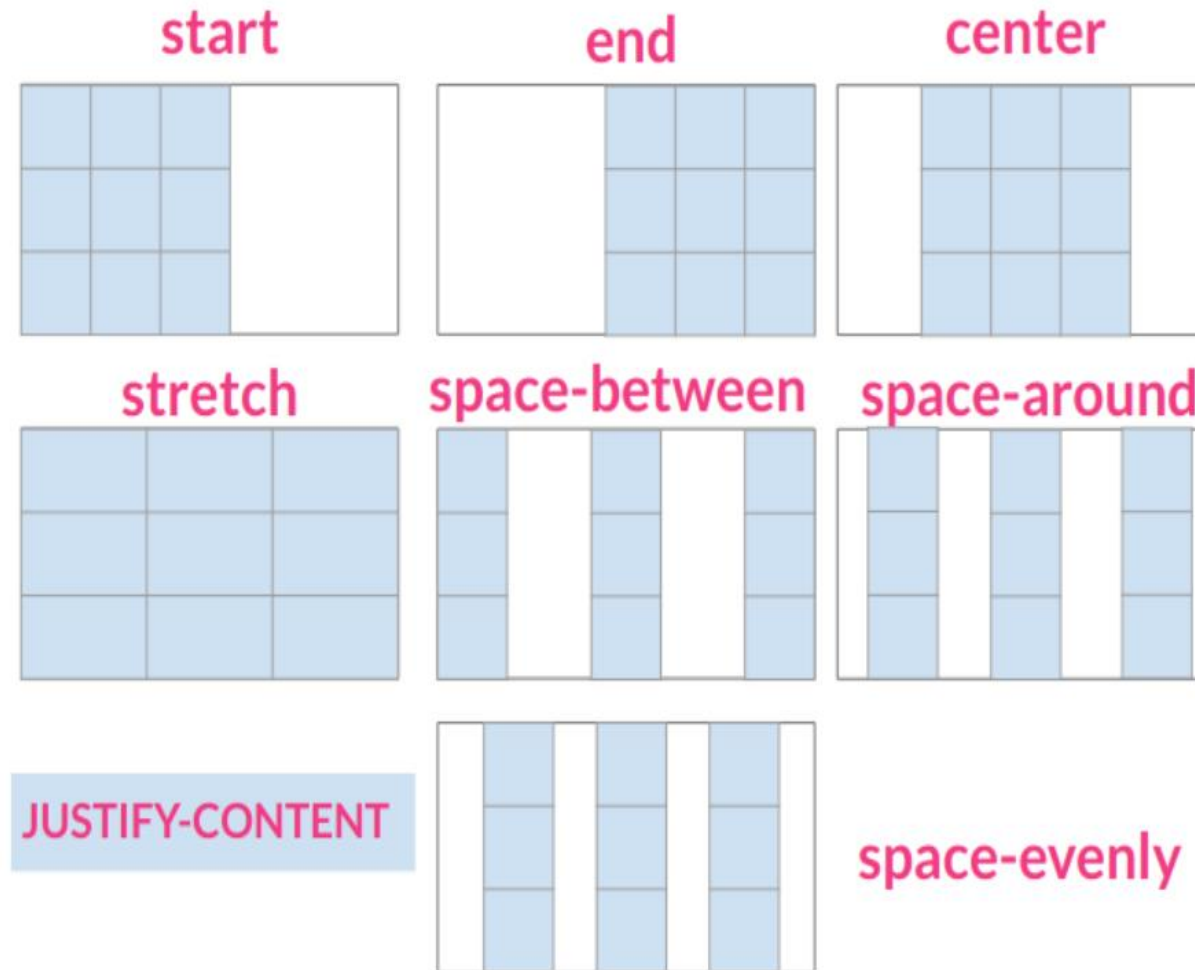
Podemos juntar estas dos propiedades usando el shorthand **place-items**, indicando primero la alineación **vertical** y luego la **horizontal**.

Alineación horizontal dentro del contenedor

Si los elementos no ocupan todo el ancho del contenedor, por defecto estos pasan a ocuparlo todo (**stretch**), pero se pueden dar otras alineaciones, dándole diferentes valores a la propiedad **justify-content**.

- **start**
- **end**
- **center**
- **stretch**
- **space-between**
- **space-around**
- **space-evenly**

Alineación horizontal

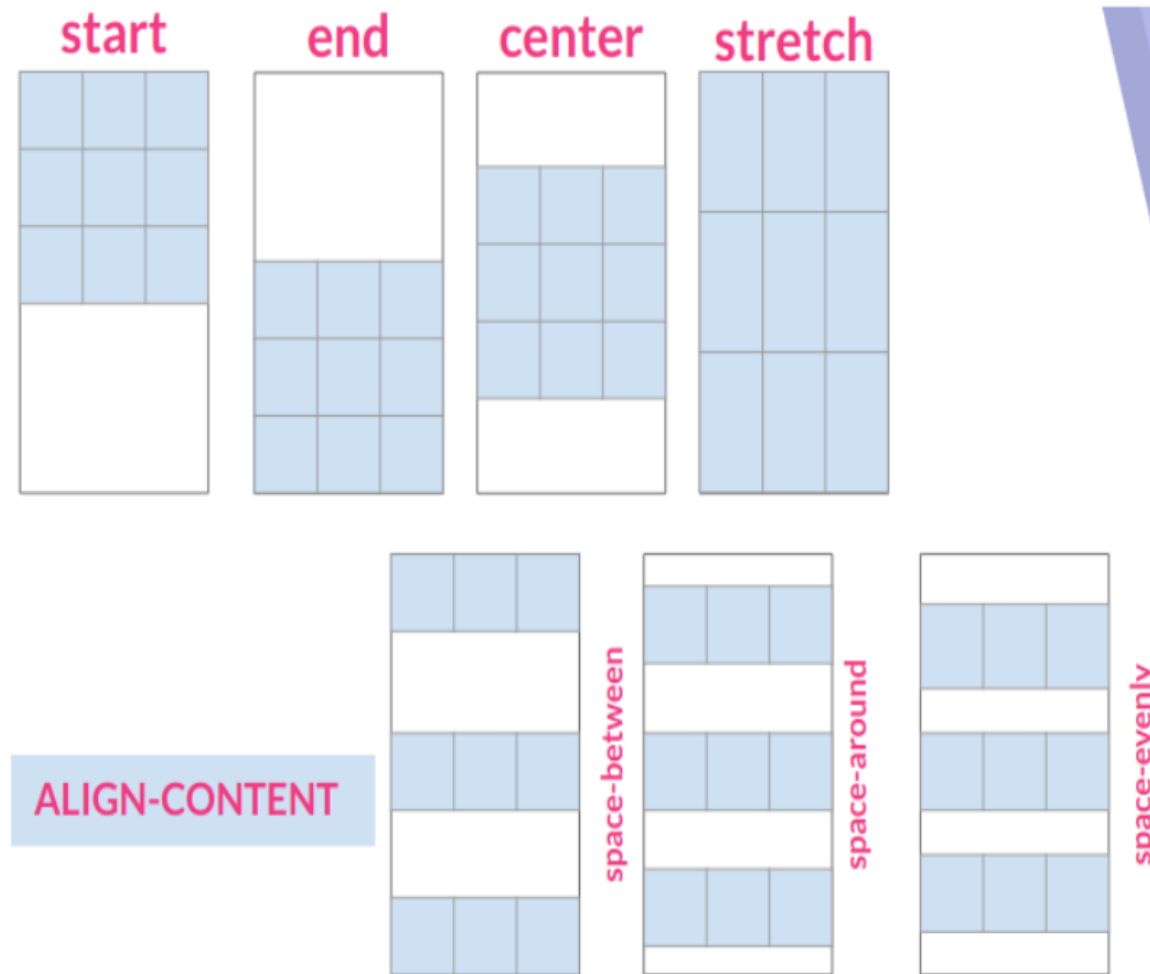


Alineación vertical dentro del contenedor

Al igual que con la alineación horizontal, por defecto los elementos del grid ocupan todo el alto del contenedor(**stretch**), pero se pueden dar otras alineaciones, dándole diferentes valores a la propiedad **align-content**.

- **start**
- **end**
- **center**
- **stretch**
- **space-between**
- **space-around**
- **space-evenly**

Alineación horizontal



Alineación dentro del contenedor

Podemos juntar estas dos propiedades usando el shorthand **place-content**, indicando primero la alineación **vertical** y luego la **horizontal**.

Propiedades de los elementos

- grid-column-start
- grid-column-end
- grid-column
- grid-row-start
- grid-row-end
- grid-row
- justify-self
- align-self
- order

Posición del elemento en la rejilla

Por defecto los elementos de dentro del contenedor, se van colocando, en el mismo orden en el que aparecen en el HTML en cada una de las celdas de la rejilla.

¿Pero si queremos que un elemento ocupe más de una celda?

Posición del elemento en la rejilla

Podemos asignar al elemento propiedades para decirle en que **columna empieza y acaba** y en que **fila empieza y acaba**.

- grid-column-start
- grid-column-end
- grid-row-start
- grid-row-end

Posición del elemento en la rejilla

Podemos usar shorthands para asignar los mismos valores

- **grid-column:** grid-column-start / grid-column-end
- **grid-row:** grid-row-start / grid-row-end
- **grid-row:** grid-row-start / grid-columna-start / grid-row-end / grid-columna-end

Ejercicio

Entrar en la siguiente página web y hacer hasta la pantalla 17.
Leer bien los textos para aprender cosas sobre el posicionamiento en la rejilla.

<https://cssgridgarden.com/#es>

Alineación horizontal

Si queremos alinear horizontalmente un elemento concreto de manera individual usaremos su propiedad **justify-self** que podrá tomar los siguientes valores:

- start
- end
- center
- stretch

Alineación vertical

Si queremos alinear verticalmente un elemento concreto de manera individual usaremos su propiedad **align-self** que podrá tomar los siguientes valores:

- start
- end
- center
- stretch

Order

Al igual que con flex, podemos usar order para cambiar el orden de los elementos. Por defecto el valor es 0.

Ejercicio

Seguir jugando hasta el final

<https://cssgridgarden.com/#es>

Posición del elemento en la rejilla

Existe otra manera de posicionar elementos en la rejilla, para eso necesitaremos definir la propiedad `grid-template-areas` en el contenedor y `grid-area` en cada uno de los elementos.

Posición del elemento en la rejilla

Usando la propiedad **grid-template-areas** en el **contenedor** asignaremos a cada celda el nombre de un área.

```
.container {  
  display: grid;  
  grid-template-columns: repeat(5,  
20%);  
  grid-template-rows: repeat(3, 100px);  
  grid-template-areas:  
    "cab cab cab cab cab"  
    "menu . main main main"  
    "menu pie pie pie pie";  
}
```

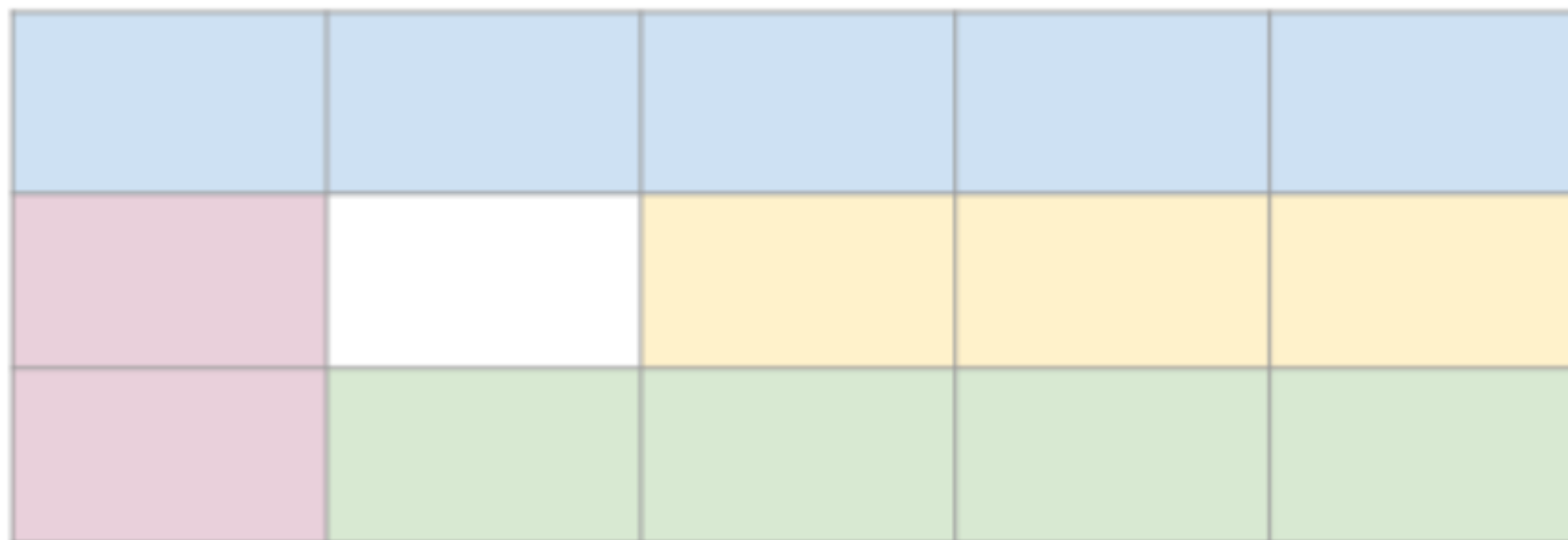
Posición del elemento en la rejilla

Los nombres de área tienen que ser los mismos que les damos a los **elementos** por medio de la propiedad **grid-area**.

```
#cab {  
  background-color: blue;  
  grid-area: cab;  
}  
  
#pie {  
  background-color: green;  
  grid-area: pie;  
}
```

```
#menu {  
  background-color: red;  
  grid-area: menu;  
}  
  
#principal {  
  background-color:  
yellow;  
  grid-area: main;  
}
```

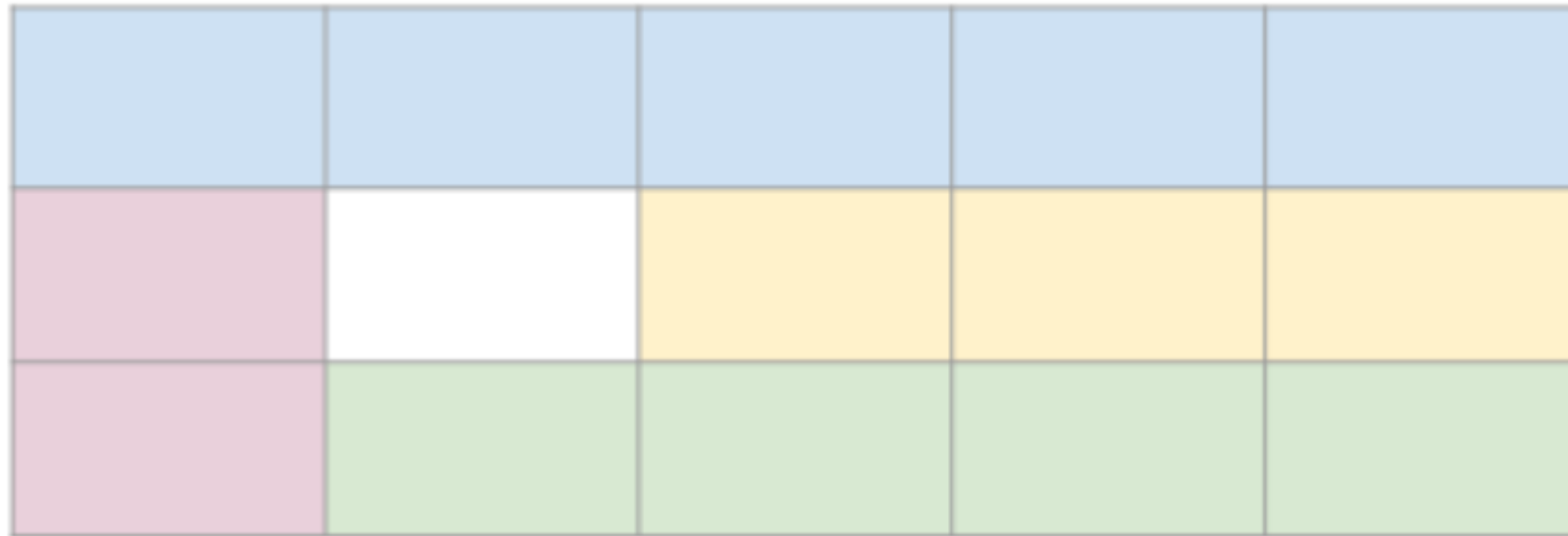
Posición del elemento en la rejilla



Otra manera de usar **grid-area**, es especificar, grid-row-start / grid-column-start / grid-row-end / grid-column-end

```
.verde{  
  grid-area: 3 / 2 / 4 / 6 ;  
}
```

Posición del elemento en la rejilla



Al definir las **grid-template-areas** aparecía un punto. Cuando hay puntos, en esos espacios se colocan los elementos que no tienen definida la propiedad **grid-area**, en el mismo orden en el que aparecen en el HTML.

Posición del elemento en la rejilla

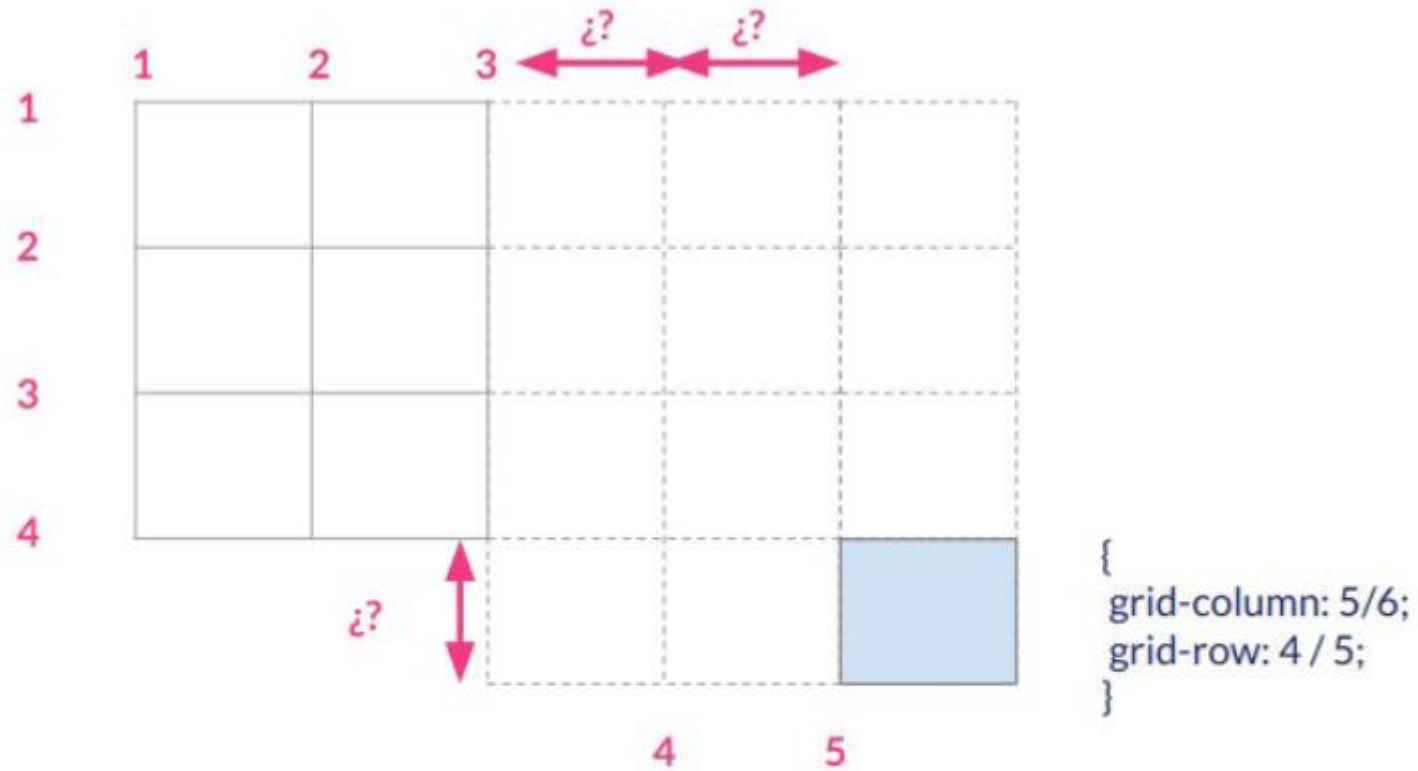
Dependiendo del valor de la propiedad **grid-auto-flow** del elemento contenedor, rellenara estos espacios no asignados teniendo en cuenta lo siguiente.

- **row:** Rellena primero las filas. Es la opción por defecto.
- **column:** Rellena primero las columnas.

Colocación implícita

Cuando un elemento queda fuera de la rejilla definida en el contenedor, podemos darle ancho a las columnas y alto a las filas que hay entre la rejilla y este elemento mediante las propiedades **grid-auto-columns** y **grid-auto-rows**

Colocación implícita



```
.container {  
  grid-auto-columns: 50px;  
  grid-auto-rows: 50px;  
}
```

} (por defecto 0)